

College of Engineering

COMP 491 – Computer Engineering Design

Project Proposal

OrderForge: A High-Fidelity Cryptocurrency Exchange Backtesting Simulator Using Historical Order Book Data

Fall 2025

Participant Information:

Name	ID	Email	Phone
Bora Çağılcı	79147	bcagilci21@ku.edu.tr	05323577991
Alp Demirel	79630	alpdemirel21@ku.edu.tr	05415185144
Tan Çağatay	79759	tcagatay21@ku.edu.tr	05312141027
Mehmet Kaan Kütük	79643	mkutuk21@ku.edu.tr	05392326540

Project Advisor: Alptekin Küpçü

Abstract

OrderForge is a high-fidelity exchange backtesting simulator that reconstructs and replays historical limit order book dynamics for a single trading pair (BTC-USD on Coinbase) using Level 3 (L3) order book data. The project is motivated by the gap between simplified backtesting assumptions and real exchange microstructure, where execution outcomes depend on order-level updates, cancellations, queue priority, and exchange-specific rules. Building on established market-making theory (e.g., Avellaneda–Stoikov), the system will provide a deterministic replay engine, a realistic matching engine enforcing price-time priority, and a strategy layer for controlled experiments. [2]

A primary objective is reliable, continuous data capture and durable storage using AWS cloud services. Market data updates will be ingested via official exchange APIs (WebSocket incremental streams plus periodic snapshots) and stored as daily time-partitioned logs in Amazon S3 with lifecycle policies and versioning. A deterministic event contract will normalize raw messages into ordered events to drive the simulator and enable verification against snapshots. [3]

The expected outcome is an end-to-end backtesting platform that (i) captures and stores an auditable dataset, (ii) reconstructs the historical order book with measurable accuracy, (iii) simulates order execution with realistic constraints, and (iv) evaluates market-making strategies with reproducible results and performance metrics.

TABLE OF CONTENTS

SECTION 1 INTRODUCTION4

1.1 CONCEPT4

1.2 OBJECTIVES4

1.3 BACKGROUND4

SECTION 2 S/T METHODOLOGY AND ASSOCIATED WORK PLAN4

2.1 METHODOLOGY4

2.2 WORK PACKAGE DESCRIPTIONS4

2.3 DEMONSTRATION5

2.4 IMPACT5

2.5 RISK ANALYSIS5

2.6 GANTT CHART5

SECTION 3 ECONOMICAL AND ETHICAL ISSUES5

SECTION 4 REFERENCES5

Section 1 Introduction

1.1 Concept

OrderForge is a modular software system that converts raw exchange market data into a deterministic simulation of an exchange matching engine, enabling realistic backtests for a single trading pair (BTC-USD on Coinbase) using Level 3 (L3) order book data. The workflow is:

1. Capture (Cloud): Continuous ingestion of incremental order book updates from the exchange WebSocket feed and periodic full snapshots from the REST "product book" endpoint. For Coinbase Exchange, the WebSocket "full" channel is explicitly designed to maintain a Level-13 (order-by-order) book when combined with snapshots. [4]

2. Replay (Deterministic): Offline reading of daily log files, normalization into a common event model, and strict ordering by exchange timestamp / sequence with explicit tie-breakers.

3. Exchange Simulation: In-memory order book and matching engine implementing price-time priority and order type rules; emits fills, rejections, acknowledgements, and book state.

4. Strategy + Evaluation: Implementation of Avellaneda–Stoikov market making to generate quotes and manage inventory risk; evaluation with consistent metrics across repeated runs. [5]

The system will be validated by comparing reconstructed book states to periodic snapshots and by deterministic reproducibility tests.

1.2 Objectives

The project objectives are measurable and aligned to a 14-week semester plan:

O1 — Cloud capture reliability: Deploy a capture service that runs unattended on AWS and records incremental updates and snapshots for a single trading pair (BTC-USD on Coinbase) using Level 3 (L3) order book data, producing daily files with audit timestamps. Success criterion: reconnection handling documented, and gaps (if any) detected and logged rather than silently ignored. Coinbase explicitly warns about slow consumption (e.g., ErrSlowConsume), motivating a non-blocking ingestion design.[6]

O2 — Deterministic replay: Define a normalized event contract and replay engine with explicit ordering rules (exchange timestamp → sequence → local timestamp → ingest index). Success criterion: identical outputs (fills, PnL, metrics) across repeated runs with the same inputs.

O3 — Verified order book reconstruction: Reconstruct book state and verify against periodic snapshots. Success criterion: at snapshot checkpoints, best bid/ask match exactly and depth comparisons meet a predefined tolerance threshold, with mismatch diagnostics.

O4 — Exchange simulator correctness: Implement matching engine with price-time priority, cancellation, partial fills, and core order types (GTC, IOC, post-only). Success criterion: unit tests covering matching invariants, regression tests on captured datasets, and deterministic outcomes.

O5 — Strategy evaluation: Implement Avellaneda–Stoikov strategy and evaluate across multiple days. Success criterion: reproducible reports with PnL, inventory distribution, spread statistics, and sensitivity analysis over parameters.[5]

1.3 Background

Limit order books are the dominant market mechanism in many electronic markets; empirical and theoretical research shows that microstructure details (order flow, cancellations, queue priority) materially affect execution outcomes.[7] A widely cited market-making formulation by Avellaneda and Stoikov provides a principled baseline for optimal quoting under inventory risk, but practical evaluation requires a realistic simulation environment.[5]

For cryptocurrency exchanges, official market data APIs can provide order-level updates. In Coinbase Exchange documentation, the WebSocket full channel is explicitly described as producing real-time updates that can be applied to a level-3 snapshot to maintain an accurate copy of the order book.[4] Coinbase also documents rate limits and operational constraints and recommends best practices (e.g., mitigating slow consumer errors), which motivates robust capture architecture and explicit backpressure handling.

A key engineering requirement is determinism: concurrent systems and network timing can introduce non-repeatability. Deterministic replay literature shows that capturing and enforcing event order is essential for reproducible execution and debugging in nondeterministic systems.[9] OrderForge applies these principles by defining strict replay ordering for market events and by isolating strategy evaluation from nondeterministic wall-clock behavior.[8]

Section 2 S/T methodology and associated work plan

2.1 Methodology

OrderForge follows an incremental, verification-first methodology:

Phase 1 — Foundations + Cloud capture: Build repository scaffold, data contracts, and a capture service capable of continuous ingestion and durable storage on AWS.

Phase 2 — Replay + reconstruction + verification: Implement deterministic replay that reconstructs the order book from incremental events; validate against snapshots at fixed intervals.

Phase 3 — Exchange simulator: Implement matching engine and account/fee tracking; integrate with replay to simulate order placement.

Phase 4 — Strategy + evaluation: Implement Avellaneda–Stoikov market making, run experiments, and produce reproducible metrics and reports.

This approach is intentionally “data-and-verification first” so that later strategy results are scientifically defensible.

Data contracts and file formats (capture output)

All captured records are stored line-delimited for streaming and simplicity:

- Incremental updates file (daily): `PAIR.incremental.YYYY-MM-DD.log`
- Snapshots file (daily): `PAIR.snapshots.YYYY-MM-DD.log`

Each line uses the required format: `<local_ts_ns>|<channel>|<raw_json>`

- `local_ts_ns`: local monotonic/UTC timestamp captured when the message was received on the EC2 host (used for latency diagnostics and audit).
- `channel`: e.g., `l3`, `snapshot`, `trades` (if needed).
- `raw_json`: verbatim message for traceability.

Snapshot policy: capture a full product book snapshot every 10 minutes. For Coinbase Exchange, the REST “Get product book” endpoint supports “Level 3” (non-aggregated) order book detail, enabling verification

checkpoints.[10]

Replay ordering rules (determinism)

Market data engine emits normalized events ordered by:

1. `exchange_ts` (if provided),
2. `seq` (exchange sequence number if provided),
3. `local_ts_ns`,
4. `ingest_index` (monotonic counter per file).

This ordering is documented and unit-tested to ensure reproducibility if information could be reached properly.

Verification tests (correctness)

- Unit tests: parsing, event ordering, book insert/cancel/update operations, matching invariants.
- Integration tests: replay a captured window and compare reconstructed book versus snapshots at 10-minute checkpoints; produce mismatch diffs.
- Performance tests: synthetic burst generator to ensure capture can avoid slow consumer conditions emphasized by Coinbase.[6]

2.2 Work Package Descriptions

Detailed work package descriptions with contributors, objectives, task descriptions, deliverables and milestones. A template is given below.

Work package number	1	**Start date or starting event:**	Week 1	
Work package title	**Repository, Contracts and Cloud Capture**			
Participant number	1	2	3	4
Participant name	Bora Çağılcı	Alp Demirel	Tan Çağatay	Kaan Kütük
Weeks per participant	4 weeks	4 weeks	4 weeks	4 weeks
Objectives				
• Define module boundaries, data contracts, event ordering rules. • Implement capture pipeline: WebSocket listener → queue → writer; periodic snapshots (10 min); daily rotation. • Deploy to AWS with durable storage and monitoring.				
Description of work				
T1.1 (w1–w2) Repo scaffold: setup, CI, formatting, module layout. T1.2 (w1–w3) Define normalized event model + on-disk schema + ordering rules; add parser tests. T1.3 (w2–w4) Capture service v1 (single connection): non-blocking ingestion; writer batching; log rotation. T1.4 (w3–w4) Snapshot scheduler (10-minute); checkpoint logging; basic gap/slow-consumer detection. T1.5 (w4) AWS deployment: systemd service, alarms, S3 upload + lifecycle + versioning.				
Deliverables				
D1.1 (w2) Architecture + data-contract docs and initial repo template.				

D1.2 (w3) Local capture prototype writing correct file format. D1.3 (w4) Cloud capture running on EC2 with daily rotation and S3 upload.
Milestones
M1 (w4) ≥6-hour continuous capture run with stable queue depth and logged reconnects. M2 (w4) Successful end-to-end restore test: download a day from S3 and verify checksums.

Work package number	2	**Start date or starting event:**	Week 5	
Work package title	**Deterministic Replay and Order Book Reconstruction**			
Participant number	1	2	3	4
Participant name	Bora Çağılcı	Alp Demirel	Tan Çağatay	Kaan Kütük
Weeks per participant	3 weeks	3 weeks	3 weeks	3 weeks
Objectives				
• Implement loader + parser + deterministic ordering. • Reconstruct LOB state and verify against snapshots. • Produce a validation report.				
Description of work				
T2.1 (w5) Implement file reader + normalized event stream API. T2.2 (w5–w7) Implement LOB reconstruction with order-level updates. T2.3 (w6–w7) Snapshot comparison framework: top-of-book and depth checks; mismatch diagnostics.				
Deliverables				
D2.1 (w6) Replay module integrated in test harness. D2.2 (w7) Verification report with mismatch statistics and example diffs.				
Milestones				
M3 (w7) Snapshot checkpoint correctness: best bid/ask matches for ≥95% checkpoints (target 99%+), with documented causes for remaining mismatches.				

Work package number	3	**Start date or starting event:**	Week 8	
Work package title	**Exchange Simulator (Matching + Account + Fees)**			
Participant number	1	2	3	4
Participant name	Bora Çağılcı	Alp Demirel	Tan Çağatay	Kaan Kütük
Weeks per participant	3 weeks	3 weeks	3 weeks	3 weeks
Objectives				
• Implement matching rules and order types (GTC/IOC/post-only). • Implement account/position tracking, fees, fill events. • Integrate with replay and event bus.				
Description of work				

T3.1 (w8–w9) Matching engine core: price-time priority; partial fills. T3.2 (w9–w10) Order APIs: submit/cancel/replace; post-only rejection logic. T3.3 (w10) Account + fee model; end-to-end deterministic regression tests.
Deliverables
D3.1 (w10) End-to-end replay + simulated order placement producing fill logs.
Milestones
M4 (w10) Determinism check: repeated runs yield identical fill streams and metrics.

Work package number	4	**Start date or starting event:**	Week 11	
Work package title	**Strategy, Experiments and Final Demo/Report**			
Participant number	1	2	3	4
Participant name	Bora Çağılcı	Alp Demirel	Tan Çağatay	Kaan Kütük
Weeks per participant	4 weeks	4 weeks	4 weeks	4 weeks
Objectives				
• Implement Avellaneda–Stoikov market making and baselines. • Run multi-day backtests and sensitivity analysis. • Produce final demo and report.				
Description of work				
T4.1 (w11) Strategy interface: callbacks, order throttling, risk limits. T4.2 (w11–w12) Avellaneda–Stoikov quoting with inventory control. [5] T4.3 (w12–w13) Experiments, parameter sweeps, metrics and plots. T4.4 (w14) Final demo preparation and report polishing.				
Deliverables				
D4.1 (w13) Experimental results pack (PnL, inventory, spread, drawdown). D4.2 (w14) Final demonstration + final written report.				
Milestones				
M5 (w14) Demo: deterministic replay + strategy run + metrics output from a selected historical day.				

2.3 Demonstration

Final demonstration will show:

1. Download of one daily dataset from S3 and replay initialization.
2. Reconstruction of the LOB and checkpoint verification against snapshots.
3. Launch of exchange simulator + strategy interface; Avellaneda–Stoikov strategy runs over historical events producing deterministic fills and account evolution.
4. Presentation of metrics: PnL time series, inventory distribution, spread, execution rates.

Determinism will be demonstrated by running the same scenario mutliple times and confirming identical outputs.

2.4 Impact

OrderForge provides a backtesting platform for:

- Understanding market microstructure through realistic LOB reconstruction (motivated by LOB research and models).[7]
- Evaluating market-making strategies in a reproducible way using established theory.
- Applying systems engineering practices (cloud reliability, monitoring, cost control) aligned with AWS Well-Architected guidance.

2.5 Risk analysis

Risk 1 — Data gaps / disconnects / slow consumption: Exchange WebSocket feeds can disconnect or throttle clients; Coinbase explicitly notes slow consumer errors and recommends best practices. Mitigation: bounded queue, separation of listener/writer, batching IO; reconnect with exponential backoff; store raw messages; gap logging.

Risk 2 — API changes / availability of true L3: Exchanges may change channel semantics or rate limits; Level-3 may be heavy. Mitigation: design abstraction to support L2 if needed; configurable data source; keep raw capture for auditing.

Risk 3 — Cloud cost overruns: Continuous EC2 + storage can exceed budget without monitoring. Mitigation: AWS Budgets alerts + forecast notifications.[11] Use CloudWatch alarms for operational signals and to trigger actions/notifications. [2] Adopt S3 lifecycle transitions/expiration for old datasets.[13]

Risk 4 — Reproducibility failures: concurrency and time ordering can cause nondeterministic outputs; deterministic replay literature highlights nondeterminism challenges.[9] Mitigation: explicit ordering contract, deterministic seed handling, strict separation of simulation clock from wall clock, regression tests.

Risk 5 — Legal/ethical constraints: Market data usage is governed by provider terms; Coinbase provides Market Data Terms of Use and developer platform terms.[14]Mitigation: comply with terms (no circumvention of rate limits, proper attribution/usage), restrict to educational/research use, avoid redistribution beyond permitted scope.

2.6 Gantt Chart

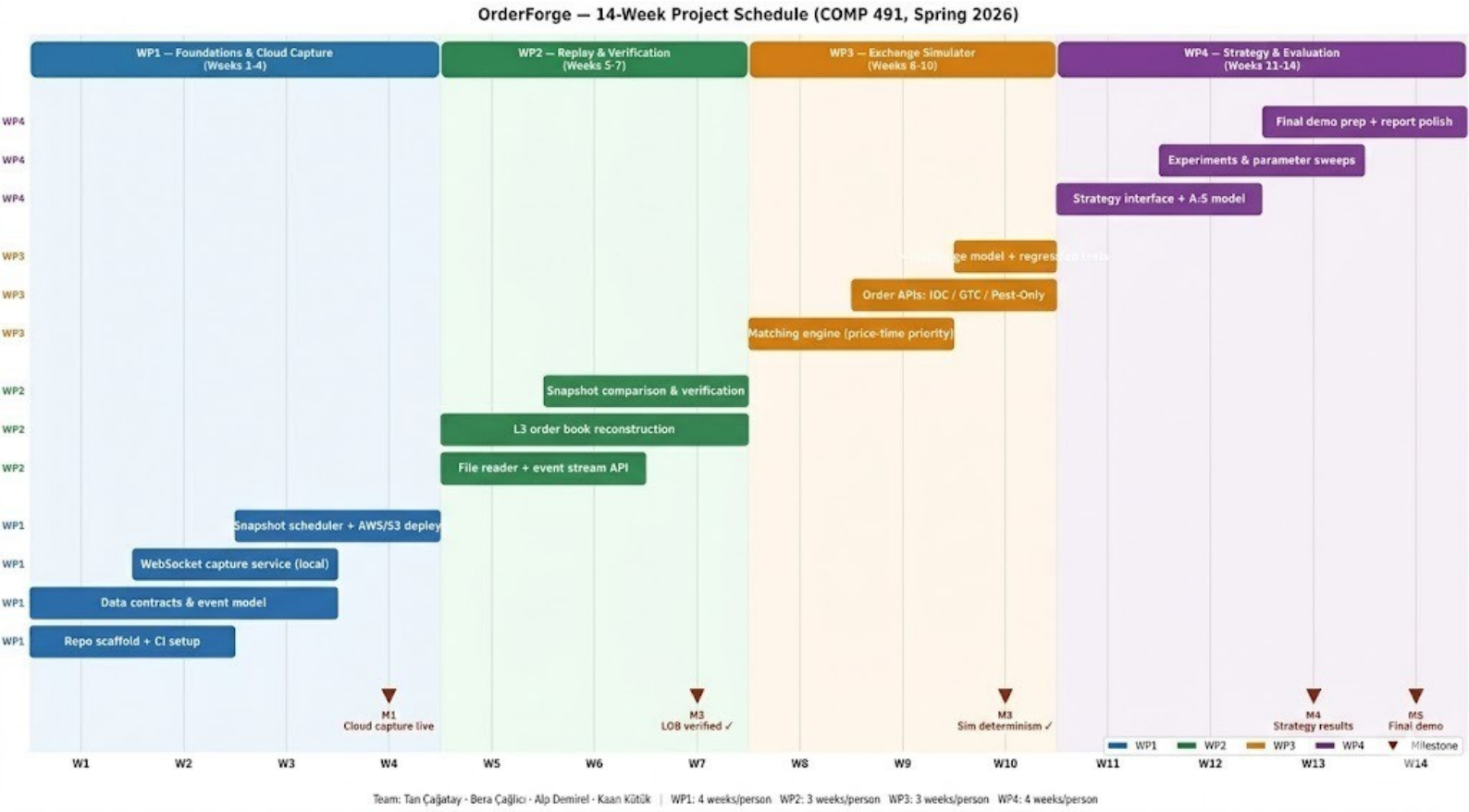


Figure 1: OrderForge 14-week Gantt chart. WP1 (blue, weeks 1–4), WP2 (green, weeks 5–7), WP3 (orange, weeks 8–10), WP4 (purple, weeks 11–14). Milestones ▼ mark key verification gates.

Section 3 Economical and Ethical Issues

OrderForge has the following economic and ethical constraints:

Economic constraints: The system requires continuous cloud compute and storage. To prevent uncontrolled spending, the project will use AWS Budgets alerts and CloudWatch alarms, and it will apply S3 lifecycle rules to transition or expire older datasets. System design choices (instance sizing, right-sizing, durable storage selection) will follow AWS Well-Architected cost optimization guidance.

Ethical constraints and engineering ethics: The capture and usage of market data must comply with provider terms. Coinbase provides both Market Data Terms of Use and developer platform terms, and also documents market data operational limitations. The project will not attempt to circumvent rate limits or restrictions and will be used strictly for educational/research purposes with transparent reporting of assumptions and limitations.

The team will follow professional ethical guidance for computing and engineering practice, including the ACM Code of Ethics and the IEEE Code of Ethics, emphasizing honesty in claims, avoidance of harm, and responsible handling of data and results.

Section 4 References

1. Avellaneda, M., & Stoikov, S. (2008). High-frequency trading in a limit order book. Quantitative Finance, 8(3), 217–224.
2. Cont, R., Stoikov, S., & Talreja, R. (2010). A stochastic model for order book dynamics (widely cited manuscript).
3. Cornell / NYU preprint version of Avellaneda–Stoikov limit-order-book market making (preprint accessible).
4. Coinbase Developer Documentation. Exchange APIs — Welcome / Introduction.
5. Coinbase Developer Documentation. Exchange WebSocket Channels — Full channel (Level-3 book maintenance).
6. Coinbase Developer Documentation. Exchange WebSocket Best Practices (slow consumer errors, subscription guidance).
7. Coinbase Developer Documentation. Exchange WebSocket Rate Limits.
8. Coinbase Developer Documentation. Get Product Book (Exchange REST API; Level 3 description).
9. Coinbase Legal. Market Data Terms of Use.
10. Coinbase Legal. Developer Platform Terms of Service.
11. Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., & Howison, S. D. (2013). Limit order books (survey). Quantitative Finance.